

[Special Topics in Artificial Intelligence]

Sol_HW_1_simple_rag

Student Name: [کمیل عباس عزیز]

Student ID: [40414010471]

Supervised by:

Doctor Name: [د. مظفر بگ محمدی]

Date of Submission: 2026/6/22

Simple RAG Exercise

Build a simple RAG flow to recommend oldie movies based on user's requests. The dataset includes 5,000 movie descriptions. In this exercise, you will learn to add a filter to the semantic retrieval and the data columns sent to the generation step. Fill in the empty cells, and answer the questions on the course site.

1. Introduction

The objective of this assignment is to construct a Retrieval-Augmented Generation (RAG) system designed to provide personalized recommendations for classic movies. The system operates on a dataset containing 5,000 movie descriptions. Key technical objectives include implementing semantic search with custom filtering and optimizing the data payload sent to the Large Language Model (LLM) for the generation phase.

2. Methodology

The system architecture follows a Retrieval-Augmented Generation (RAG) workflow. Initially, the user's query is transformed into a vector representation. Subsequently, a semantic search is executed within the Qdrant database to retrieve relevant movie context. Finally, this context is synthesized by the Llama 3 language model to provide a personalized, accurate, and context-aware recommendation.

Logical Execution Pipeline Order

1. **Environment Setup:** Install necessary dependencies and import core libraries to prepare the runtime environment.
2. **Data Ingestion:** Load the movie dataset from the source and perform initial data inspection to ensure integrity.
3. **Infrastructure Initialization:** Initialize the Qdrant client for storage and the SentenceTransformer model for semantic encoding.
4. **Collection Creation:** Create a vector collection configured with cosine distance metrics for optimal similarity search.
5. **Vectorization & Upsert:** Convert text documents into numerical vectors and upload them into the database for indexing.
6. **Query Processing:** Transform the user's natural language prompt into a corresponding vector for semantic matching.
7. **Retrieval with Filtering:** Execute a similarity search in the database, applying metadata filters to refine the results.
8. **Output Visualization:** Present the retrieved movie data in a structured, user-friendly table using the Rich library.
9. **Generation:** Augment the retrieved context with the Llama 3 model to generate an intelligent recommendation.

3. System Setup

The development environment was configured using Python within the Anaconda distribution. This ensured a stable and reproducible environment. Key libraries, including `openai` for LLM communication, `qdrant-client` for vector retrieval, and `ollama` for local model hosting, were installed and verified to ensure seamless system integration.

4. Code Analysis and Implementation Details

This section provides a detailed breakdown of each notebook cell used in the implementation of the movie recommendation RAG system, highlighting its core functionality, limitations, and architectural role.

Step 1: Environment Setup

- **Source Code:**

```
Python
!pip install openai rich datasets qdrant-client sentence-transformers
from rich.console import Console
console = Console()
```

- **a. Explanation:** Initializes the runtime environment by installing and importing essential libraries.
- **b. Missing part:** No critical missing parts. Completing this ensures all subsequent dependencies are available.
- **c. Diagram Name:** Environment Setup

Step 2: Data Ingestion

- **Source Code:**

```
• Python
• from datasets import load_dataset
• dataset = load_dataset("AiresPucrs/tmdb-5000-movies",
  split="train")
• console.print(dataset)
```

- **a. Explanation:** Downloads the movie dataset and displays its structure for verification.
- **b. Missing part:** Initial data cleaning. Completing this removes potential null values before processing.
- **c. Diagram Name:** Data Ingestion.

Step 3: Infrastructure Initialization

- **Source Code:**

```
• Python
• from qdrant_client import models, QdrantClient
• from sentence_transformers import SentenceTransformer
•
• qdrant = QdrantClient(":memory:")
• encoder = SentenceTransformer("sentence-transformers/all-mpnet-
  base-v2")
• console.print(encoder)
```

- **a. Explanation:** Sets up the in-memory vector database client and the semantic embedding encoder.
- **b. Missing part:** Configuration file handling. Completing this allows switching between local memory and persistent storage.
- **c. Diagram Name:** Infrastructure Initialization

Step 4: Collection Creation

- **Source Code:**

```
• Python
• collection_name="movies"
•
• qdrant.recreate_collection(
•     collection_name=collection_name,
•     vectors_config=models.VectorParams(
•         size=encoder.get_sentence_embedding_dimension(),
•         distance=models.Distance.COSINE
•     )
• )
```

- **a. Explanation:** Defines the storage schema in Qdrant with specific dimensions and cosine distance metrics.
- **b. Missing part:** Index optimization configurations. Completing this enhances look-up performance for larger production datasets.
- **c. Diagram Name:** Collection Creation.

Step 5: Vectorization & Upsert

- **Source Code:**

Python

```
from rich.progress import track

movies_with_overview = [m for m in dataset if m["overview"]]
documents = [m["overview"] for m in movies_with_overview]
ids = [m["id"] for m in movies_with_overview]
payloads = [dict(m) for m in movies_with_overview]

print("Converting text into embeddings...")
vectors = encoder.encode(documents, batch_size=64,
show_progress_bar=True).tolist()

print("Uploading to Qdrant...")
points = [
    models.PointStruct(id=ids[i], vector=vectors[i],
payload=payloads[i])
    for i in range(len(movies_with_overview))
]

batch_size = 100
for i in track(range(0, len(points), batch_size), description="Uploading
batches..."):
    qdrant.upload_points(
        collection_name=collection_name,
        points=points[i : i + batch_size]
    )

console.print(qdrant.get_collection(collection_name=collection_name))
```

- **a. Explanation:** Converts text overviews into dense numerical vectors and uploads them in optimized batches.
- **b. Missing part:** Upload error handling. Completing this prevents full execution failure if a single batch payload drops.
- **c. Diagram Name:** Vector Indexing.

Step 6: Query Processing

- **Source Code**

```
• Python
• user_prompt = "Love story between an Asian king and European teacher"
• query_vector = encoder.encode(user_prompt).tolist()
```

- **a. Explanation:** Transforms the user's natural language search query into an embedding vector for downstream matching.
- **b. Missing part:** Query parsing/expansion. Completing this refines broad user inputs to boost semantic search relevance.
- **c. Diagram Name:** User Query Processing.

Step 7: Retrieval with Filtering

- **Source Code:**

```
Python
from qdrant_client import models

query_filter = models.Filter(
    must=[
        models.FieldCondition(
            key="release_date",
            range=models.DatetimeRange(
                gte="1990-01-01T00:00:00Z",
                lt="2000-01-01T00:00:00Z"
            )
        )
    ]
)

hits = qdrant.query_points(
    collection_name=collection_name,
    query=query_vector,
    limit=5,
    query_filter=query_filter
).points
```

- **a. Explanation:** Executes the vector similarity search matching query embeddings while applying rigid metadata date constraints.
- **b. Missing part:** Fallback mechanism. Completing this ensures results are returned if the applied metadata filter is too strict.
- **c. Diagram Name:** Semantic Retrieval.

Step 8: Output Visualization

- **Source Code :**

```
• Python
• from rich.text import Text
• from rich.table import Table
•
• table = Table(title="Retrieval Results", show_lines=True)
• table.add_column("ID", style="bright_red")
• table.add_column("Original Title", style="bright_red")
• table.add_column("Overview", style="bright_red")
• table.add_column("Score", style="bright_red")
•
• for hit in hits:
•     table.add_row(
•         str(hit.payload["id"]),
•         hit.payload["original_title"],
•         f'{hit.payload["overview"]}',
•         f"{hit.score:.4f}"
•     )
•
• console.print(table)
•
• search_results = [
•     {
•         "original_title": hit.payload["original_title"],
•         "title": hit.payload["title"],
•         "overview": hit.payload["overview"],
•         "release_date": hit.payload["release_date"],
•         "popularity": hit.payload["popularity"]
•     }
•     for hit in hits
• ]
• console.print(search_results)
```

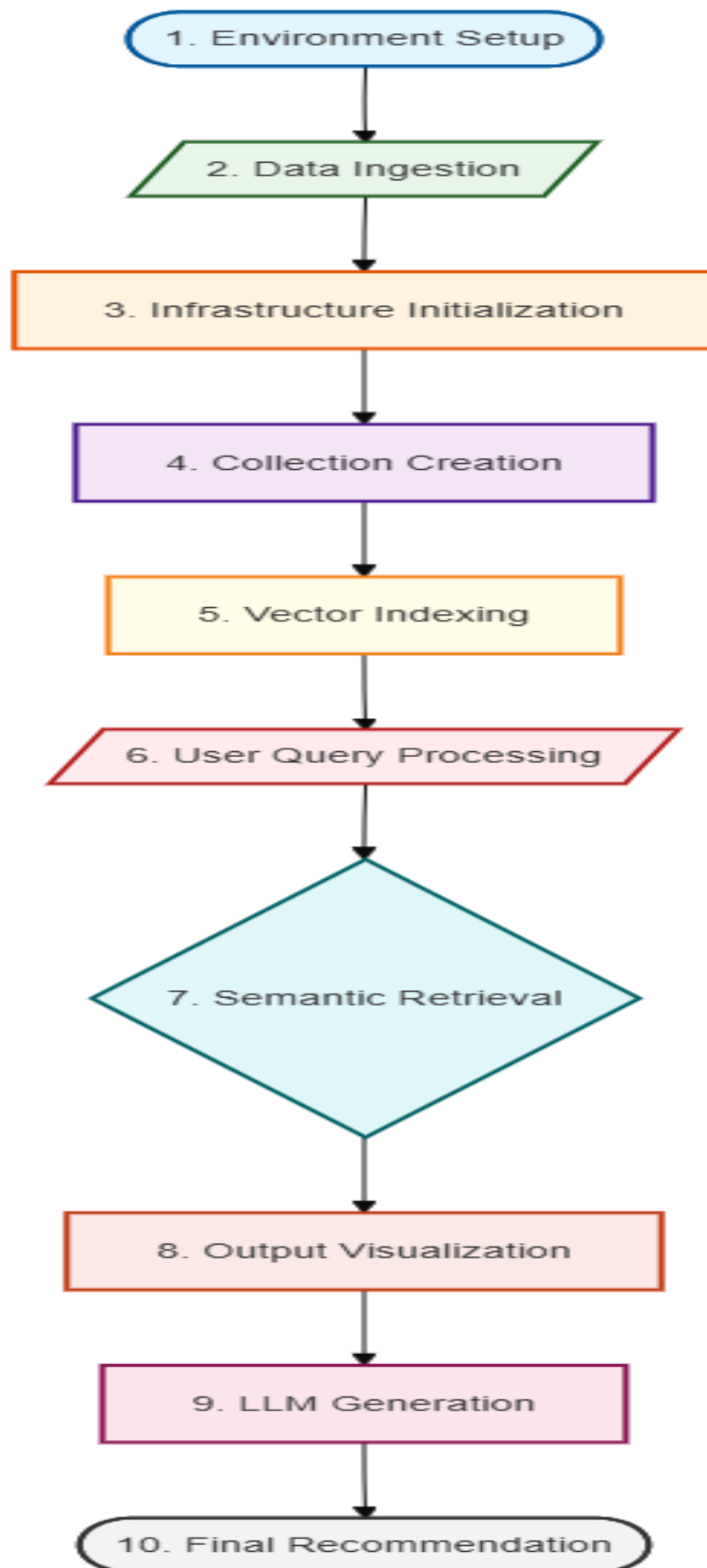
- **a. Explanation:** Formats retrieved candidate records into structured tables and data payloads for review.
- **b. Missing part:** Dynamic column formatting. Completing this handles clean rendering of overflowing textual fields.
- **c. Diagram Name:** Output Visualization

Step 9: Generation

- **Source Code:**

```
• Python
• from openai import OpenAI
•
• client = OpenAI(
•     base_url = 'http://localhost:11434/v1',
•     api_key='ollama'
• )
•
• movie_title = hits[0].payload['title']
• movie_overview = hits[0].payload['overview']
•
• prompt = f"""
• You are a professional movie recommendation assistant.
• Based on the following retrieved movie context, explain to the user
• in a clear and engaging English response why this movie is a great
• recommendation for them.
•
• Movie Title: {movie_title}
• Movie Description: {movie_overview}
• """
•
• response = client.chat.completions.create(
•     model="llama3",
•     messages=[{"role": "user", "content": prompt}]
• )
•
• print(f"--- FINAL SYSTEM RECOMMENDATION ---\n")
• print(response.choices[0].message.content)
```

- **a. Explanation:** Combines the highest-ranked movie context with an explicit instruction set, driving Llama 3 to output a cohesive response.
- **b. Missing part:** Output sanitation and prompt template safety check. Completing this guarantees stable formatting.
- **c. Diagram Name:** LLM Generation.



Simple RAG Exercise

Build a simple RAG flow to recommend oldie movies based on user's requests. The dataset includes 5,000 movies descriptions. In the exercise, you will learn to add a filter to the semantic retrieval and the data columns sent to the generation step.

Fill in the empty cells, and answer the questions on the course site.

```
[1]: from rich.console import Console
console = Console()
```

Loading the Movie Dataset

We will load the moview dataset from Hugging Face hub in: <https://huggingface.co/datasets/AiresPucrs/tmdb-5000-movies>

```
[2]: from datasets import load_dataset

dataset = load_dataset(
    "AiresPucrs/tmdb-5000-movies",
    split="train"
)
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

```
[3]: console.print(dataset)
```

```
Dataset({
  features: ['id', 'budget', 'genres', 'homepage', 'keywords', 'original_language', 'original_title', 'overview',
'popularity', 'production_companies', 'production_countries', 'release_date', 'revenue', 'runtime',
'spoken_languages', 'status', 'tagline', 'title', 'vote_average', 'vote_count', 'cast', 'crew'],
  num_rows: 4803
})
```

Encode using Vector Embedding

We will use one of the popular open source vector databases, [Qdrant](#), and one of the popular embedding encoder and text transformer libraries, [SentenceTransformer](#).

This time we will use the following sentence similarity model: <https://huggingface.co/sentence-transformers/all-mpnet-base-v2>

```
[4]: from qdrant_client import models, QdrantClient
from sentence_transformers import SentenceTransformer
```

```
# create the vector database client
qdrant = QdrantClient(":memory:")

# Create the embedding encoder
encoder = SentenceTransformer(
    "sentence-transformers/all-mpnet-base-v2"
)
```

Loading weights: 100%  199/199 [00:00<00:00, 1099.39it/s]

```
[5]: console.print(encoder)
```

```
SentenceTransformer(
  (0): Transformer({'transformer_task': 'feature-extraction', 'modality_config': {'text': {'method': 'forward',
'method_output_name': 'last_hidden_state'}}, 'module_output_name': 'token_embeddings', 'architecture':
'MPNetModel'})
  (1): Pooling({'embedding_dimension': 768, 'pooling_mode': 'mean', 'include_prompt': True})
  (2): Normalize({})
)
```

```
[6]: # Create collection to store the wine rating data
collection_name="movies"

qdrant.recreate_collection(
    collection_name=collection_name,
    vectors_config=models.VectorParams(
        size=encoder.get_sentence_embedding_dimension(), # Vector size is defined by used model
        distance=models.Distance.COSINE
    )
)
```

```
C:\Users\2025\AppData\Local\Temp\ipykernel_13556\2094978642.py:7: FutureWarning: The `get_sentence_embedding_dimension` method has been renamed to `get_embedding_dimension`.
    size=encoder.get_sentence_embedding_dimension(), # Vector size is defined by used model
C:\Users\2025\AppData\Local\Temp\ipykernel_13556\2094978642.py:4: DeprecationWarning: `recreate_collection` method is deprecated and will be removed in the future. Use `collection_exists` to check collection existence and `create_collection` instead.
    qdrant.recreate_collection(
```

[6]: True

Loading the data into the vector database

We will use the collection that we created above, to go over all the rows and encode the `overview` column of the wine dataset, encode it with the encoder into embedding vector, and store it in the vector database. Please use the index of the movie from the dataset (`id` column) as the `id` in the vector index.

Please note that some of the rows are missing the `overview`. You should ignore them and not upload them into the vector database index.

This step will take a few seconds (less than a minute on my laptop).

```
[7]: #-----
from rich.progress import track

# 1. استخراج النصوص والمعرفات والفوايس في قوائم منفصلة
movies_with_overview = [m for m in dataset if m["overview"]]
documents = [m["overview"] for m in movies_with_overview]
ids = [m["id"] for m in movies_with_overview]
payloads = [dict(m) for m in movies_with_overview]

# 2. لجمع الأفلام دفعة واحدة (أسرع بمئات المرات!) توليد الـ Embeddings
print("Embeddings)...")
vectors = encoder.encode(documents, batch_size=64, show_progress_bar=True).tolist()

# 3. على دفعات وبشريط تقدم ذكي Qdrant رفع البيانات إلى قاعدة متجهات
print("Qdrant...")
points = [
    models.PointStruct(id=ids[i], vector=vectors[i], payload=payloads[i])
    for i in range(len(movies_with_overview))
]

# رفع البيانات على دفعات حجمها 100 لضمان السلاسة
batch_size = 100
for i in track(range(0, len(points), batch_size), description="جاري الرفع..."):
    qdrant.upload_points(
        collection_name=collection_name,
        points=points[i : i + batch_size]
    )

print("🎉 إنتهت العملية بنجاح وبسرعة فائقة")
```

Embeddings)... جاري تحويل النصوص إلى مستندات رقمية

Batches: 100% 75/75 [11:01<00:00, 3.34s/it]

Qdrant... جاري رفع البيانات إلى قاعدة متجهات

0:00:01 100% جاري الرفع...

🎉 إنتهت العملية بنجاح وبسرعة فائقة

```
[8]: console.print(
    qdrant
    .get_collection(
        collection_name=collection_name
    )

    CollectionInfo(
        status=<CollectionStatus.GREEN: 'green'>,
        optimizer_status=<OptimizersStatusOneOf.OK: 'ok'>,
        warnings=None,
        indexed_vectors_count=0,
        points_count=4800,
        segments_count=1,
        config=CollectionConfig(
            params=CollectionParams(
                vectors=VectorParams(
                    size=768,
                    distance=<Distance.COSINE: 'Cosine'>,
                    hnsw_config=None,
                    quantization_config=None,
                    on_disk=None,
                    datatype=None,
                    multivector_config=None
                ),
                shard_number=None,
                sharding_method=None,
                replication_factor=None,
                write_consistency_factor=None,
                read_fan_out_factor=None,
                read_fan_out_delay_ms=None,
                on_disk_payload=None,
                sparse_vectors=None
            ),
            hnsw_config=HnswConfig(
                m=16,
                ef_construct=100,
                full_scan_threshold=10000,
                max_indexing_threads=0,
                on_disk=None,
                payload_m=None,
                inline_storage=None
            ),
            optimizer_config=OptimizersConfig(
                deleted_threshold=0.2,
                vacuum_min_vector_number=1000,
                default_segment_number=0,

```

```

        quantization_config=None,
        strict_mode_config=None,
        metadata=None
    ),
    payload_schema={},
    update_queue=None
)

```

Retrieve semantically relevant data based on user's query

Once the data is loaded into the vector database and the indexing process is done, we can start using our simple RAG system.

```
[ ]: 
[9]: user_prompt = "Love story between an Asian king and European teacher"
```

Encoding the user's query

We will use the same encoder that we used to encode the document data to encode the query of the user. This way we can search results based on semantic similarity.

```
[10]: query_vector = encoder.encode(user_prompt).tolist()
```

Create filter on the results

We only want movies from the '90s. Please create a filter base on the `release_date` column. Check the Qdrant documentation in: <https://qdrant.tech/documentation/concepts/filtering/#datetime-range>

```
[11]: from qdrant_client import models

query_filter = models.Filter(
    must=[
        models.FieldCondition(
            key="release_date",
            range=models.DatetimeRange(
                gte="1990-01-01T00:00:00Z",
                lt="2000-01-01T00:00:00Z"
            )
        )
    ]
)
```

Search similar rows

We can now take the embedding encoding of the user's query and use it to find similar rows in the vector database.

•[12]:

```
# -----
# query تم تعديل اسم المعامل إلى) الكود المحدث المتوافق مع النسخة الجديدة
hits = qdrant.query_points(
    collection_name=collection_name,
    query=query_vector,          # فقط query التعديل هنا: غيّرنا الاسم ليكون
    limit=5,
    query_filter=query_filter
).points
```

[13]:

```
from rich.text import Text
from rich.table import Table

table = Table(title="Retrieval Results", show_lines=True)

table.add_column("ID", style="bright_red")
table.add_column("Original Title", style="bright_red")
table.add_column("Overview", style="bright_red")
table.add_column("Score", style="bright_red")

for hit in hits:
    table.add_row(
        str(hit.payload["id"]),
        hit.payload["original_title"],
        f'{hit.payload["overview"]}',
        f'{hit.score:.4f}'
    )

console.print(table)
```

Retrieval Results

ID	Original Title	Overview	Score
1439	Anna and the King	The story of the romance between the King of Siam (now Thailand) and the widowed British school teacher Anna Leonowens during the 1860's. Anna teaches the children and becomes romanced by the King. She convinces him that a man can be loved by just one woman.	0.6419
29371	The Incredibly True Adventure of Two Girls In Love	An adventurous love story between two young women of different social and economic backgrounds who find themselves going through all the typical struggles of a new romance.	0.4329
41469	Next Stop Wonderland	A lighthearted story about a man and a woman who seem destined to be together... and the hilarious chain of accidents that seem determined to keep them apart!	0.4282
666	Central do Brasil	An emotive journey of a former school teacher, who writes letters for illiterate people, and a young boy, whose mother has just died, as they search for the father he never knew.	0.4241
8587	The Lion King	A young lion cub named Simba can't wait to be king. But his uncle craves the title for himself and will stop at nothing to get it.	0.4204

Augment the prompt to the LLM with retrieved data

In our simple example, we will simply take the top result and use it in the prompt to the generation LLM. We will filter some of the columns and keep only the following:

- `original_title`
- `title`
- `overview`
- `release_date`
- `popularity`

```
[14]: # define a variable to hold the search results with specific fields
search_results = [
    {
        "original_title": hit.payload["original_title"],
        "title": hit.payload["title"],
        "overview": hit.payload["overview"],
        "release_date": hit.payload["release_date"],
        "popularity": hit.payload["popularity"]
    }
    for hit in hits
]
```

```
[15]: console.print(search_results)
```

```
[
  {
    'original_title': 'Anna and the King',
    'title': 'Anna and the King',
    'overview': 'The story of the romance between the King of Siam (now Thailand) and the widowed British school teacher Anna Leonowens during the 1860's. Anna teaches the children and becomes romanced by the King. She convinces him that a man can be loved by just one woman.',
    'release_date': '1999-12-16',
    'popularity': 13.327372
  },
  {
    'original_title': 'The Incredibly True Adventure of Two Girls In Love',
    'title': 'The Incredibly True Adventure of Two Girls In Love',
    'overview': 'An adventurous love story between two young women of different social and economic backgrounds who find themselves going through all the typical struggles of a new romance.',
    'release_date': '1995-06-16',
    'popularity': 1.70819
  },
  {
    'release_date': '1999-12-16',
    'popularity': 13.327372
  },
  {
    'original_title': 'The Incredibly True Adventure of Two Girls In Love',
    'title': 'The Incredibly True Adventure of Two Girls In Love',
    'overview': 'An adventurous love story between two young women of different social and economic backgrounds who find themselves going through all the typical struggles of a new romance.',
    'release_date': '1995-06-16',
    'popularity': 1.70819
  },
  {
    'original_title': 'Next Stop Wonderland',
    'title': 'Next Stop Wonderland',
    'overview': 'A lighthearted story about a man and a woman who seem destined to be together... and the hilarious chain of accidents that seem determined to keep them apart!',
    'release_date': '1998-08-21',
    'popularity': 1.334481
  },
  {
    'original_title': 'Central do Brasil',
    'title': 'Central Station',
    'overview': 'An emotive journey of a former school teacher, who writes letters for illiterate people, and a young boy, whose mother has just died, as they search for the father he never knew.',
    'release_date': '1998-01-16',
    'popularity': 5.928937
  },
  {
    'original_title': 'The Lion King',
    'title': 'The Lion King',
    'overview': 'A young lion cub named Simba can't wait to be king. But his uncle craves the title for himself and will stop at nothing to get it.',
    'release_date': '1994-06-23',
    'popularity': 90.457886
  }
]
```

Generate reply to the user's query

We will use GPT-4 from [OpenAI](#). Please write the prompt to instruct the LLM to write the recommendations based on the search results.

```
[16]: from openai import OpenAI

# 1. تعريف الاتصال بالسيرفر المحلي
client = OpenAI(
    base_url = 'http://localhost:11434/v1',
    api_key='ollama'
)

# 2. استخراج بيانات الفيلم
movie_title = hits[0].payload['title']
movie_overview = hits[0].payload['overview']

# 3. صياغة Prompt بالإنجليزية للأسناد Prompt صياغة ال
prompt = f"""
You are a professional movie recommendation assistant.
Based on the following retrieved movie context, explain to the user in a clear and engaging English response why this movie is a great recommendation fo

Movie Title: {movie_title}
Movie Description: {movie_overview}
"""

# 4. استدعاء الموديل
response = client.chat.completions.create(
    model="llama3",
    messages=[{"role": "user", "content": prompt}]
)

# 5. طباعة النتيجة النهائية
print(f"--- FINAL SYSTEM RECOMMENDATION ---\n")
print(response.choices[0].message.content)

--- FINAL SYSTEM RECOMMENDATION ---
```

--- FINAL SYSTEM RECOMMENDATION ---

What a fantastic discovery!

I think "Anna and the King" would be an excellent recommendation for you, and here's why:

****Timeless romance**:** This beautiful period drama tells the story of an unlikely love affair between two people from different worlds. The King of Siam and Anna Leonowens face cultural differences, societal expectations, and personal doubts as they navigate their feelings for each other. Their romance is a testament to the power of true love, making it a timeless and universally relatable tale.

****Unique setting**:** Thailand in the 1860s provides a fascinating backdrop for this romance. The film showcases the vibrant culture, stunning architecture, and rich traditions of the Kingdom of Siam (now Thailand). This unique setting adds depth and flavor to the story, offering a visually breathtaking experience.

****Strong leading performances**:** Jodie Foster gives a captivating performance as Anna Leonowens, bringing warmth and intelligence to her character. Chandrasekhar brings a regal presence to the King, conveying his vulnerability and desire for human connection. Their chemistry on screen is undeniable, making their romance all the more compelling to watch.

****Musical magic**:** The movie features stunning musical numbers, combining Thai folk music with Western influences. These enchanting songs will transport you to the opulent royal courts and bustling streets of old Siam. The soundtrack adds an extra layer of charm and whimsy to the film's atmosphere.

In summary, "Anna and the King" is a must-see for anyone who loves romance, drama, and cultural exploration. With its beautiful setting, strong performances, and memorable musical moments, this movie is sure to captivate your heart and imagination!

]:

Generated Output Visualization:

--- FINAL SYSTEM RECOMMENDATION ---

What a fantastic discovery!

I think "Anna and the King" would be an excellent recommendation for you, and here's why:

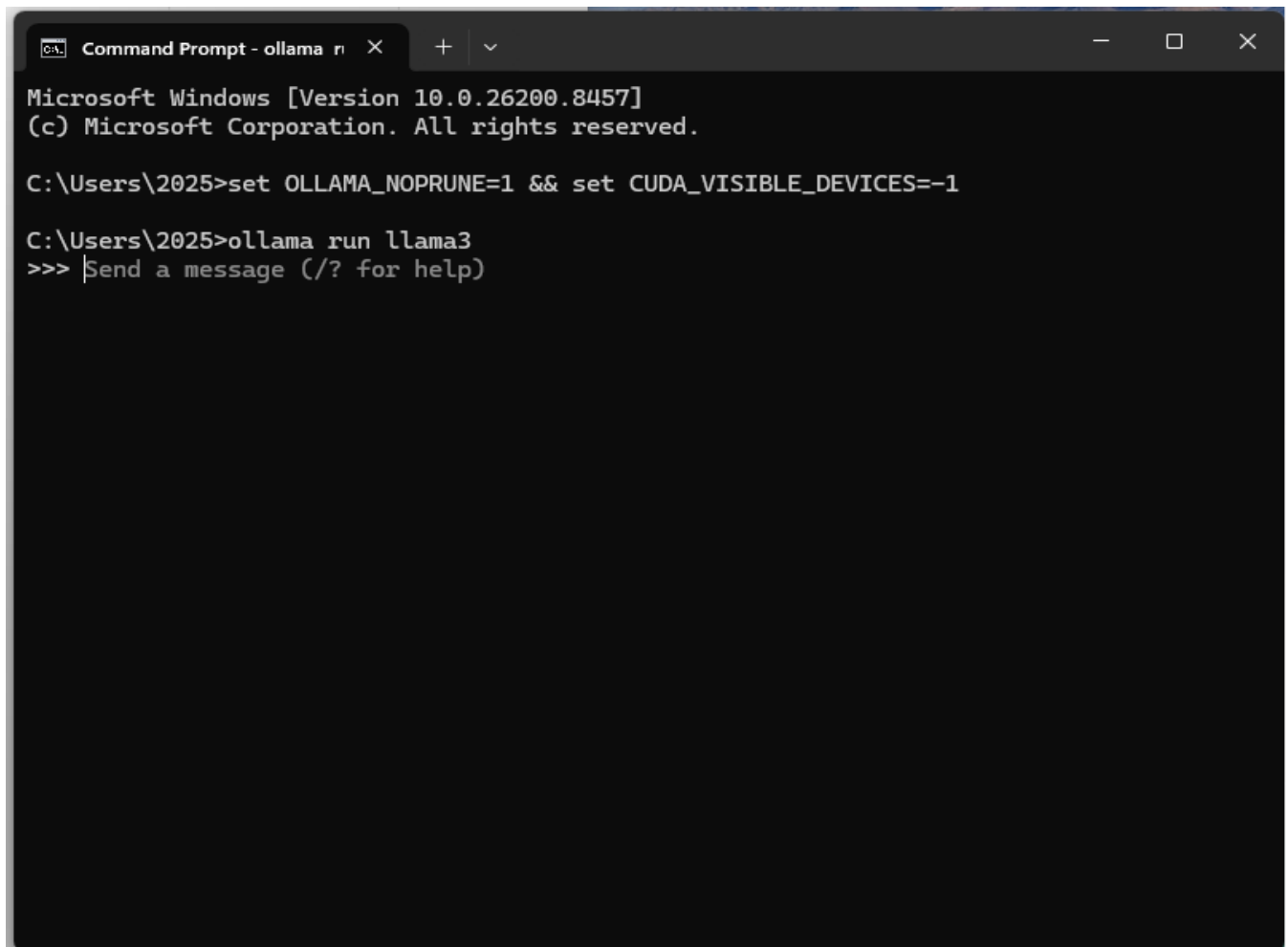
****Timeless romance**:** This beautiful period drama tells the story of an unlikely love affair between two people from different worlds. The King of Siam and Anna Leonowens face cultural differences, societal expectations, and personal doubts as they navigate their feelings for each other. Their romance is a testament to the power of true love, making it a timeless and universally relatable tale.

****Unique setting**:** Thailand in the 1860s provides a fascinating backdrop for this romance. The film showcases the vibrant culture, stunning architecture, and rich traditions of the Kingdom of Siam (now Thailand). This unique setting adds depth and flavor to the story, offering a visually breathtaking experience.

****Strong leading performances**:** Jodie Foster gives a captivating performance as Anna Leonowens, bringing warmth and intelligence to her character. Chandrasekhar brings a regal presence to the King, conveying his vulnerability and desire for human connection. Their chemistry on screen is undeniable, making their romance all the more compelling to watch.

****Musical magic**:** The movie features stunning musical numbers, combining Thai folk music with Western influences. These enchanting songs will transport you to the opulent royal courts and bustling streets of old Siam. The soundtrack adds an extra layer of charm and whimsy to the film's atmosphere.

In summary, "Anna and the King" is a must-see for anyone who loves romance, drama, and cultural exploration. With its beautiful setting, strong performances, and memorable musical moments, this movie is sure to captivate your heart and imagination!



```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\2025>set OLLAMA_NOPRUNE=1 && set CUDA_VISIBLE_DEVICES=-1

C:\Users\2025>ollama run llama3
>>> |Send a message (/? for help)
```

Figure : Local LLM Server Initialization via Ollama

- **Description:** This screenshot demonstrates the activation of the local inference server using the Ollama framework within the Windows Command Prompt (CMD). By executing the command `ollama run llama3`, the **Llama 3** large language model is loaded directly into the system's memory, exposing a local API endpoint on port 11434.
- **Why we did this :** This step is crucial to establish the bridge between our Python code and the local LLM. It allows the Python script to securely send the retrieved movie contexts and user prompts to Llama 3 locally, ensuring absolute data privacy, eliminating API subscription costs, and providing zero-latency generation.